SAVEETHA SCHOOL OF ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

Name: K.B.Devarshan.
Reg.no: 192511426

Course Code:CSA1709
Course Name:Artificial Intelligence

Exp :21 Write a Prolog Program to implement Towers of Hanoi.

Aim: To Write a Prolog Program to implement Towers of Hanoi.

Algorithm:
Step 1: Define the main rule to initiate the problem, specifying the initial number of disks, the starting source peg (left), the final destination peg (right), and the auxiliary peg (center).
Step 2: Define the base case where moving exactly 1 disk requires simply transferring it directly from the source peg to the destination peg and displaying the move.
Step 3: Define the recursive rule for moving N disks (where N is greater than 1).
Step 4: Calculate the value of N-1 to represent the smaller stack of disks above the largest disk.
Step 5: Recursively move the top N-1 disks from the source peg to the auxiliary peg, utilizing the destination peg as a temporary holder.
Step 6: Transfer the remaining 1 (largest) disk directly from the source peg to the destination peg and display the action.
Step 7: Recursively move the N-1 disks from the auxiliary peg to the final destination peg, utilizing the source peg as a temporary holder.

PROGRAM:

```
hanoi(N) :-
    move(N, left, right, center).

% Base Case: Moving 1 disk is simple. Just move it from Source to Destination.
move(1, Source, Destination, _) :-
    write('Move top disk from '),
    write(Source),
    write(' to '),
    write(Destination),
    nl.

% Recursive Step: Moving N disks (where N > 1).
move(N, Source, Destination, Auxiliary) :-
    N > 1,
    M is N - 1,
    % Step 1: Move N-1 disks from Source to Auxiliary peg
    move(M, Source, Auxiliary, Destination),

    % Step 2: Move the remaining 1 (largest) disk from Source to Destination
    move(1, Source, Destination, _),

    % Step 3: Move the N-1 disks from the Auxiliary peg to the Destination peg
    move(M, Auxiliary, Destination, Source).
```

OUTPUT:

RESULT: Thus, the program was executed and found that the output was found to be correct.

Exp :22 Write a Prolog Program to print particular bird can fly or not. Incorporate required queries.

Aim: To write a Prolog program to determine whether a particular bird can fly or not by defining facts, exceptions, and logical rules.

Algorithm:

Step 1: Define a set of facts declaring various entities as birds (e.g., sparrow, penguin, eagle).
Step 2: Define another set of facts declaring the specific exceptions, i.e., birds that are known to be flightless.
Step 3: Define a logical rule can_fly(X) which states that an entity X can fly if it is a bird AND it is not listed as flightless.
Step 4: Load the knowledge base into the Prolog compiler.
Step 5: Execute queries passing the name of the bird to determine its flying status based on the defined rules.

Program:

```
bird(sparrow).
bird(eagle).
bird(parrot).
bird(pigeon).
bird(penguin).
bird(ostrich).
bird(kiwi).

flightless(penguin).
flightless(ostrich).
flightless(kiwi).

can_fly(X) :-
```

```
    bird(X),
    \+ flightless(X).
```

Output:

RESULT: Thus, the program was executed and found that the output was found to be correct.

Exp :23 Write the prolog program to implement family tree.

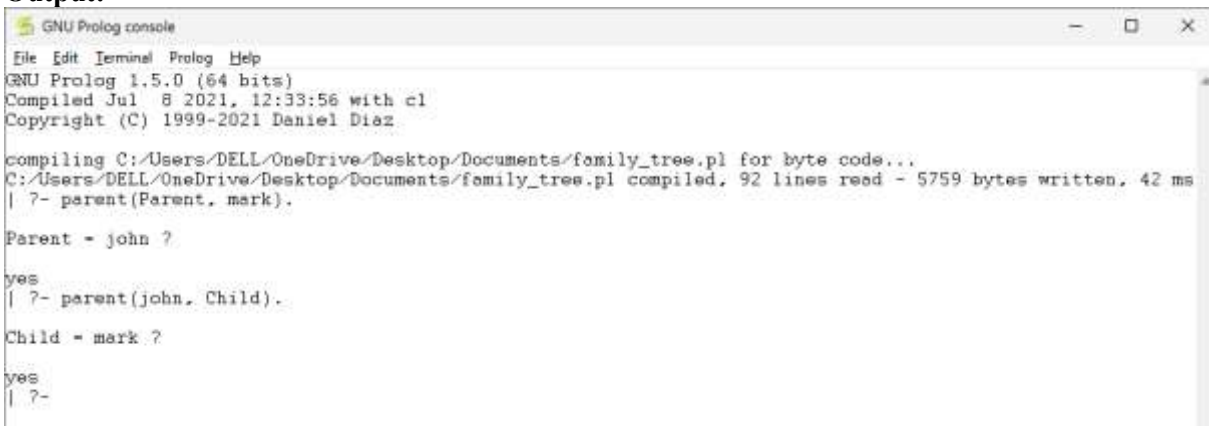
Aim:
To Write the prolog program to implement family tree.
Algorithm:
Step 1: Define basic facts to establish the gender of individuals using male(X) and female(X).
Step 2: Define basic facts to establish direct parent-child relationships using parent(X, Y) where X is the parent of Y.
Step 3: Define a logical rule for father(X, Y) specifying that X must be male and X must be a parent of Y.
Step 4: Define a logical rule for mother(X, Y) specifying that X must be female and X must be a parent of Y.
Step 5: Define a logical rule for sibling(X, Y) specifying that X and Y must share a common parent Z, and X must not be the same person as Y.
Step 6: Define a logical rule for grandparent(X, Y) by stating that X is the parent of Z, and Z is the parent of Y.
Step 7: Define additional specific rules such as brother, sister, grandfather, and grandmother by combining gender facts with the relationship rules.
Step 8: Load the program into the Prolog environment and execute queries to infer relationships from the knowledge base.
Program:

```
%Define males, females, and parent-child relationships

% male(Person)
male(john).
male(mark).
male(bob).
male(tom).
male(peter).
```

```
% female(Person)
female(mary).
female(susan).
female(alice).
female(linda).
female(jane).

% parent(Parent, Child)
% 1st Generation (John and Mary are parents of Mark and Susan)
parent(john, mark).
parent(john, susan).
parent(mary, mark).
parent(mary, susan).

% 2nd Generation, Branch 1 (Mark and Alice are parents of Bob and Linda)
parent(mark, bob).
parent(mark, linda).
parent(alice, bob).
parent(alice, linda).

% 2nd Generation, Branch 2 (Tom and Susan are parents of Peter and Jane)
parent(tom, peter).
parent(tom, jane).
parent(susan, peter).
parent(susan, jane).


%
================================================================================
% RULES: Define relationships based on facts
%
================================================================================

% Father: A person is a father if they are a parent and male.
father(F, C) :-
   parent(F, C),
   male(F).

% Mother: A person is a mother if they are a parent and female.
mother(M, C) :-
   parent(M, C),
   female(M).

% Child/Son/Daughter
child(C, P) :-
   parent(P, C).
son(S, P) :-
   parent(P, S),
   male(S).
daughter(D, P) :-
   parent(P, D),
   female(D).

% Sibling: A and B are siblings if they share the same parent, and A is not B.
sibling(A, B) :-
```

```
    parent(P, A),
    parent(P, B),
    A \= B.

% Brother/Sister
brother(B, S) :-
    sibling(B, S),
    male(B).
sister(S, B) :-
    sibling(S, B),
    female(S).

% Grandparents
grandparent(G, C) :-
    parent(G, P),
    parent(P, C).
grandfather(G, C) :-
    grandparent(G, C),
    male(G).
grandmother(G, C) :-
    grandparent(G, C),
    female(G).

% Uncle/Aunt
uncle(U, C) :-
    parent(P, C),
    brother(U, P).
aunt(A, C) :-
    parent(P, C),
    sister(A, P).
```

Output:



RESULT: Thus, the program was executed and found that the output was found to be correct.

Exp :24 Write a Prolog Program to suggest Dieting System based on Disease.

Aim:

To write a Prolog program that suggests an appropriate dieting system and dietary restrictions based on a patient's diagnosed disease.

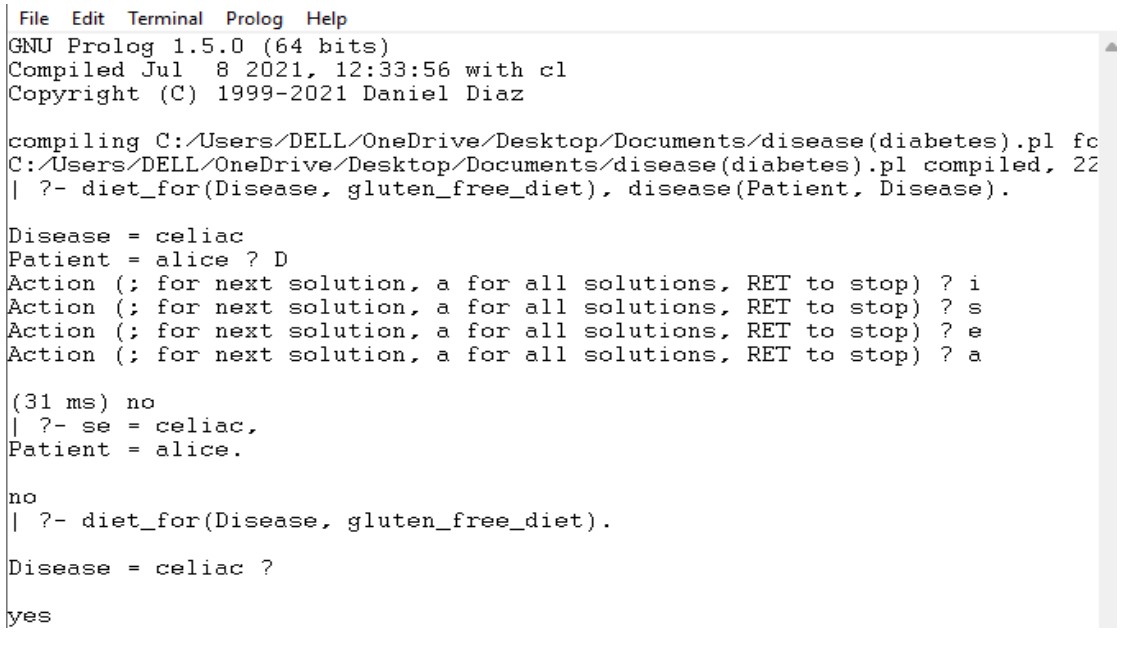
Algorithm:

Step 1: Define facts specifying the diseases diagnosed for different patients.
Step 2: Define facts detailing the recommended diet plans corresponding to various diseases.
Step 3: Define facts listing the specific foods to avoid for each respective disease.
Step 4: Create a logical rule that links a patient to their disease, the recommended diet, and the restricted foods.
Step 5: Load the code into the Prolog compiler.
Step 6: Execute queries to retrieve customized dietary recommendations for a specific patient or disease.

Program:

```
disease(john, diabetes).
disease(mary, hypertension).
disease(alice, celiac).
disease(bob, obesity).

diet_for(diabetes, low_carbohydrate_and_sugar_free).
diet_for(hypertension, low_sodium_and_dash_diet).
diet_for(celiac, gluten_free_diet).
diet_for(obesity, low_calorie_and_high_protein).

avoid_for(diabetes, sweets_and_sodas).
avoid_for(hypertension, salty_foods_and_processed_meats).
avoid_for(celiac, wheat_barley_and_rye).
avoid_for(obesity, junk_food_and_sugary_drinks).

suggest_diet(Patient, Disease, Diet, Avoid) :-
   disease(Patient, Disease),
   diet_for(Disease, Diet),
   avoid_for(Disease, Avoid).

diet_by_disease(Disease, Diet, Avoid) :-
   diet_for(Disease, Diet),
   avoid_for(Disease, Avoid).
```

OUTPUT:



```
File  Edit  Terminal  Prolog  Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

compiling C:/Users/DELL/OneDrive/Desktop/Documents/disease(diabetes).pl fo
C:/Users/DELL/OneDrive/Desktop/Documents/disease(diabetes).pl compiled, 22
| ?- diet_for(Disease, gluten_free_diet), disease(Patient, Disease).

Disease = celiac
Patient = alice ? D
Action (; for next solution, a for all solutions, RET to stop) ? i
Action (; for next solution, a for all solutions, RET to stop) ? s
Action (; for next solution, a for all solutions, RET to stop) ? e
Action (; for next solution, a for all solutions, RET to stop) ? a

(31 ms) no
| ?- se = celiac,
Patient = alice.

no
| ?- diet_for(Disease, gluten_free_diet).

Disease = celiac ?

yes
```

Exp :25 Write a Prolog program to implement Monkey Banana Problem

Aim:

To write a Prolog program to solve the Monkey and Banana problem by defining states and possible moves to reach the goal state.

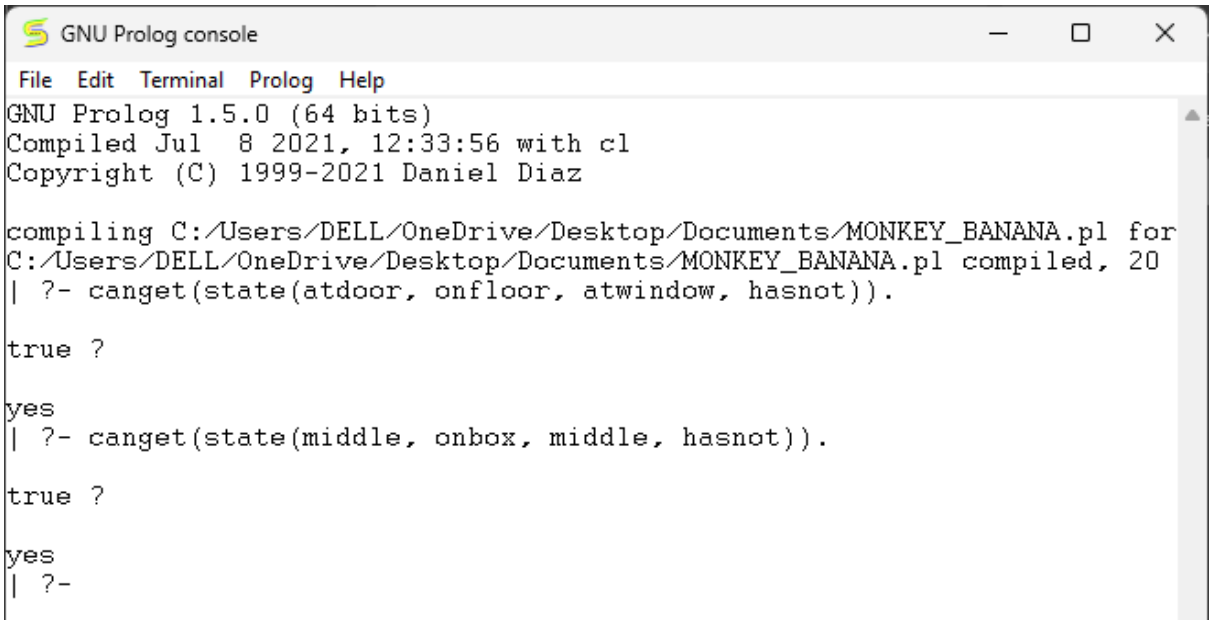
Algorithm:

Step 1: Define the state structure as state(MonkeyPosition, MonkeyElevation, BoxPosition, HasBanana).
Step 2: Establish the goal state where the monkey has grasped the banana state(_, _, _, has).
Step 3: Define the move rule for the "grasp" action, allowing the monkey to grasp the banana if it is on the box at the middle of the room.
Step 4: Define the move rule for the "climb" action, allowing the monkey to step onto the box if both are at the same location on the floor.
Step 5: Define the move rule for the "push" action, allowing the monkey to push the box from one location to another if they are together on the floor.
Step 6: Define the move rule for the "walk" action, allowing the monkey to move independently from one location to another while on the floor.
Step 7: Create a recursive canget rule that checks if the goal state is already achieved.
Step 8: If the goal is not achieved, the canget rule tries a valid move to transition into a new state and recursively calls itself until it reaches the goal.

Program:

```
% State representation: state(MonkeyPos, MonkeyElev, BoxPos, HasBanana)

% 1. The monkey can grasp the banana if it is on the box in the middle of the room.
move(state(middle, onbox, middle, hasnot), grasp, state(middle, onbox, middle, has)).

% 2. The monkey can climb onto the box if they are at the same position.
move(state(Position, onfloor, Position, HasBanana), climb, state(Position, onbox, Position,
HasBanana)).

% 3. The monkey can push the box from P1 to P2 if they are at the same position.
move(state(P1, onfloor, P1, HasBanana), push(P1, P2), state(P2, onfloor, P2, HasBanana)).

% 4. The monkey can walk from P1 to P2.
move(state(P1, onfloor, BoxPos, HasBanana), walk(P1, P2), state(P2, onfloor, BoxPos,
HasBanana)).

% Base Case: The monkey already has the banana.
canget(state(_, _, _, has)).

% Recursive Step: The monkey applies a valid move to reach a new state, and repeats.
canget(State1) :-
   move(State1, _, State2),
   canget(State2).
```

OUTPUT:



RESULT: Thus, the program was executed and found that the output was found to be correct.

Exp :26 Exp :24 Write a Prolog Program to suggest Dieting System based on Disease.

Aim:

To write a Prolog program that maps various fruits to their respective colors and demonstrates the concept of backtracking to retrieve multiple solutions for a single query.

Algorithm:

Step 1: Define a series of facts in the knowledge base linking a specific fruit to its color using the structure color(Fruit, Color).

Step 2: Create multiple facts sharing the same color (e.g., apple is red, cherry is red) to allow for multiple possible matches.

Step 3: Define a basic rule fruit_of_color(Fruit, Color) that queries the color facts.

Step 4: Load the program into the Prolog compiler.

Step 5: Execute a query containing a variable (e.g., asking for all fruits that are 'red').

Step 6: Prolog finds the first matching fact and binds the variable. When the user requests alternative solutions (using ;), Prolog applies backtracking—it undoes the variable binding, goes back up the search tree, and looks for the next matching fact until all possibilities are exhausted.

Program:

```
color(apple, red).
color(banana, yellow).
color(strawberry, red).
color(grape, purple).
color(cherry, red).
color(lemon, yellow).
color(watermelon, green).
color(lime, green).

fruit_of_color(Fruit, Color) :-
   color(Fruit, Color).
```

OUTPUT:

RESULT: Thus, the program was executed and found that the output was found to be correct.